

Activity__ Course 5 Automatidata project lab

October 11, 2025

1 Automatidata project

Course 5 - Regression Analysis: Simplify complex data relationships

The data consulting firm Automatidata has recently hired you as the newest member of their data analytics team. Their newest client, the NYC Taxi and Limousine Commission (New York City TLC), wants the Automatidata team to build a multiple linear regression model to predict taxi fares using existing data that was collected over the course of a year. The team is getting closer to completing the project, having completed an initial plan of action, initial Python coding work, EDA, and A/B testing.

The Automatidata team has reviewed the results of the A/B testing. Now it's time to work on predicting the taxi fare amounts. You've impressed your Automatidata colleagues with your hard work and attention to detail. The data team believes that you are ready to build the regression model and update the client New York City TLC about your progress.

A notebook was structured and prepared to help you in this project. Please complete the following questions.

2 Course 5 End-of-course project: Build a multiple linear regression model

In this activity, you will build a multiple linear regression model. As you've learned, multiple linear regression helps you estimate the linear relationship between one continuous dependent variable and two or more independent variables. For data science professionals, this is a useful skill because it allows you to consider more than one variable against the variable you're measuring against. This opens the door for much more thorough and flexible analysis to be completed.

Completing this activity will help you practice planning out and building a multiple linear regression model based on a specific business need. The structure of this activity is designed to emulate the proposals you will likely be assigned in your career as a data professional. Completing this activity will help prepare you for those career moments.

The purpose of this project is to demonstrate knowledge of EDA and a multiple linear regression model

The goal is to build a multiple linear regression model and evaluate the model *This activity has three parts:*

Part 1: EDA & Checking Model Assumptions * What are some purposes of EDA before constructing a multiple linear regression model?

Part 2: Model Building and evaluation * What resources do you find yourself using as you complete this stage?

Part 3: Interpreting Model Results

- What key insights emerged from your model(s)?
- What business recommendations do you propose based on the models built?

3 Build a multiple linear regression model

4 PACE stages

Throughout these project notebooks, you'll see references to the problem-solving framework PACE. The following notebook components are labeled with the respective PACE stage: Plan, Analyze, Construct, and Execute.

4.1 PACE: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

4.1.1 Task 1. Imports and loading

Import the packages that you've learned are needed for building linear regression models.

```
[7]: # Imports
# Packages for numerics + dataframes
### YOUR CODE HERE ###

import pandas as pd
import numpy as np

# Packages for visualization
### YOUR CODE HERE ###

import matplotlib.pyplot as plt
import seaborn as sns

# Packages for date conversions for calculating trip durations
### YOUR CODE HERE ###

from datetime import date
from datetime import datetime
```

```

from datetime import timedelta

# Packages for OLS, MLR, confusion matrix
### YOUR CODE HERE ###

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import sklearn.metrics as metrics
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error

```

Note: Pandas is used to load the NYC TLC dataset. As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```

[8]: # Load dataset into dataframe
df0=pd.read_csv("2017_Yellow_Taxi_Trip_Data.csv")

```

4.2 PACE: Analyze

In this stage, consider the following question where applicable to complete your code response:

- What are some purposes of EDA before constructing a multiple linear regression model?
1. Outliers and extreme data values can significantly impact linear regression equations. After visualizing data, make a plan for addressing outliers by dropping rows, substituting extreme data with average data, and/or removing data values greater than 3 standard deviations.
 2. EDA activities also include identifying missing data to help the analyst make decisions on their exclusion or inclusion by substituting values with data set means, medians, and other similar methods.
 3. It's important to check for things like multicollinearity between predictor variables, as well to understand their distributions, as this will help you decide what statistical inferences can be made from the model and which ones cannot.
 4. Additionally, it can be useful to engineer new features by multiplying variables together or taking the difference from one variable to another. For example, in this dataset you can create a `duration` variable by subtracting `tpep_dropoff` from `tpep_pickup` time.

4.2.1 Task 2a. Explore data with EDA

Analyze and discover data, looking for correlations, missing data, outliers, and duplicates.

Start with `.shape` and `.info()`.

```

[9]: # Start with `.shape` and `.info()`
### YOUR CODE HERE ###

```

```
df = df0.copy()
print(df.shape)
print(df.info())
```

```
(22699, 18)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            22699 non-null  int64
1   VendorID                             22699 non-null  int64
2   tpep_pickup_datetime                 22699 non-null  object
3   tpep_dropoff_datetime                22699 non-null  object
4   passenger_count                      22699 non-null  int64
5   trip_distance                        22699 non-null  float64
6   RatecodeID                           22699 non-null  int64
7   store_and_fwd_flag                  22699 non-null  object
8   PULocationID                        22699 non-null  int64
9   DOLocationID                        22699 non-null  int64
10  payment_type                         22699 non-null  int64
11  fare_amount                          22699 non-null  float64
12  extra                               22699 non-null  float64
13  mta_tax                             22699 non-null  float64
14  tip_amount                           22699 non-null  float64
15  tolls_amount                         22699 non-null  float64
16  improvement_surcharge                22699 non-null  float64
17  total_amount                         22699 non-null  float64
dtypes: float64(8), int64(7), object(3)
memory usage: 3.1+ MB
None
```

Check for missing data and duplicates using `.isna()` and `.drop_duplicates()`.

```
[10]: # Check for missing data and duplicates using .isna() and .drop_duplicates()
      ### YOUR CODE HERE ###
      print("Shape of dataframe with null values: ",df.drop_duplicates().shape)
      df.isna().sum()
```

Shape of dataframe with null values: (22699, 18)

```
[10]: Unnamed: 0      0
      VendorID      0
      tpep_pickup_datetime  0
      tpep_dropoff_datetime  0
      passenger_count      0
      trip_distance      0
      RatecodeID      0
```

```

store_and_fwd_flag    0
PULocationID          0
DOLocationID          0
payment_type          0
fare_amount           0
extra                 0
mta_tax               0
tip_amount            0
tolls_amount          0
improvement_surcharge 0
total_amount          0
dtype: int64

```

Use .describe().

```

[11]: # Use .describe()
      ### YOUR CODE HERE ###
      df.describe().T

```

```

[11]:
count      mean      std      min  \
Unnamed: 0  22699.0  5.675849e+07  3.274493e+07  12127.0
VendorID    22699.0  1.556236e+00  4.968384e-01      1.0
passenger_count  22699.0  1.642319e+00  1.285231e+00      0.0
trip_distance  22699.0  2.913313e+00  3.653171e+00      0.0
RatecodeID    22699.0  1.043394e+00  7.083909e-01      1.0
PULocationID  22699.0  1.624124e+02  6.663337e+01      1.0
DOLocationID  22699.0  1.615280e+02  7.013969e+01      1.0
payment_type  22699.0  1.336887e+00  4.962111e-01      1.0
fare_amount   22699.0  1.302663e+01  1.324379e+01    -120.0
extra         22699.0  3.332746e-01  4.630966e-01     -1.0
mta_tax       22699.0  4.974448e-01  3.946499e-02     -0.5
tip_amount    22699.0  1.835781e+00  2.800626e+00      0.0
tolls_amount  22699.0  3.125415e-01  1.399212e+00      0.0
improvement_surcharge 22699.0  2.995506e-01  1.567274e-02     -0.3
total_amount  22699.0  1.631050e+01  1.609730e+01    -120.3

      25%      50%      75%      max
Unnamed: 0  28520556.00  56731504.00  85374524.00  1.134863e+08
VendorID      1.00      2.00      2.00  2.000000e+00
passenger_count  1.00      1.00      2.00  6.000000e+00
trip_distance    0.99      1.61      3.06  3.396000e+01
RatecodeID      1.00      1.00      1.00  9.900000e+01
PULocationID   114.00     162.00     233.00  2.650000e+02
DOLocationID   112.00     162.00     233.00  2.650000e+02
payment_type    1.00      1.00      2.00  4.000000e+00
fare_amount      6.50      9.50     14.50  9.999900e+02
extra           0.00      0.00      0.50  4.500000e+00

```

mta_tax	0.50	0.50	0.50	5.000000e-01
tip_amount	0.00	1.35	2.45	2.000000e+02
tolls_amount	0.00	0.00	0.00	1.910000e+01
improvement_surcharge	0.30	0.30	0.30	3.000000e-01
total_amount	8.75	11.80	17.80	1.200290e+03

4.2.2 Task 2b. Convert pickup & dropoff columns to datetime

```
[12]: # Check the format of the data
      ### YOUR CODE HERE ###
      df['tpep_dropoff_datetime'] = pd.to_datetime(df['tpep_dropoff_datetime'],
      ↪format='%m/%d/%Y %I:%M:%S %p')
      df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'],
      ↪format='%m/%d/%Y %I:%M:%S %p')

      print(" date type of tpep_dropoff_datetime is: ", df['tpep_dropoff_datetime'].
      ↪dtype)
      print(" date type of tpep_pickup_datetime is: ", df['tpep_pickup_datetime'].
      ↪dtype)

      display(df.head())
```

date type of tpep_dropoff_datetime is: datetime64[ns]

date type of tpep_pickup_datetime is: datetime64[ns]

	Unnamed: 0	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	\
0	24870114	2	2017-03-25 08:55:43	2017-03-25 09:09:47	
1	35634249	1	2017-04-11 14:53:28	2017-04-11 15:19:58	
2	106203690	1	2017-12-15 07:26:56	2017-12-15 07:34:08	
3	38942136	2	2017-05-07 13:17:59	2017-05-07 13:48:14	
4	30841670	2	2017-04-15 23:32:20	2017-04-15 23:49:03	

	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	\
0	6	3.34	1	N	
1	1	1.80	1	N	
2	1	1.00	1	N	
3	1	3.70	1	N	
4	1	4.37	1	N	

	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	\
0	100	231	1	13.0	0.0	0.5	
1	186	43	1	16.0	0.0	0.5	
2	262	236	1	6.5	0.0	0.5	
3	188	97	1	20.5	0.0	0.5	
4	4	112	2	16.5	0.5	0.5	

	tip_amount	tolls_amount	improvement_surcharge	total_amount
0	2.76	0.0	0.3	16.56
1	4.00	0.0	0.3	20.80
2	1.45	0.0	0.3	8.75
3	6.39	0.0	0.3	27.69
4	0.00	0.0	0.3	17.80

```
[ ]: # Convert datetime columns to datetime
    ### YOUR CODE HERE ###
    # already converted
```

4.2.3 Task 2c. Create duration column

Create a new column called `duration` that represents the total number of minutes that each taxi ride took.

```
[13]: # Create `duration` column
    ### YOUR CODE HERE ###
    df['duration'] = (df['tpep_dropoff_datetime'] - df['tpep_pickup_datetime']) / \
        np.timedelta64(1, 'm')
    df.head()
```

```
[13]: Unnamed: 0  VendorID  tpep_pickup_datetime  tpep_dropoff_datetime  \
0      24870114         2  2017-03-25 08:55:43  2017-03-25 09:09:47
1      35634249         1  2017-04-11 14:53:28  2017-04-11 15:19:58
2      106203690         1  2017-12-15 07:26:56  2017-12-15 07:34:08
3      38942136         2  2017-05-07 13:17:59  2017-05-07 13:48:14
4      30841670         2  2017-04-15 23:32:20  2017-04-15 23:49:03
```

	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	\
0	6	3.34	1	N	
1	1	1.80	1	N	
2	1	1.00	1	N	
3	1	3.70	1	N	
4	1	4.37	1	N	

	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	\
0	100	231	1	13.0	0.0	0.5	
1	186	43	1	16.0	0.0	0.5	
2	262	236	1	6.5	0.0	0.5	
3	188	97	1	20.5	0.0	0.5	
4	4	112	2	16.5	0.5	0.5	

	tip_amount	tolls_amount	improvement_surcharge	total_amount	duration
0	2.76	0.0	0.3	16.56	14.066667
1	4.00	0.0	0.3	20.80	26.500000

2	1.45	0.0	0.3	8.75	7.200000
3	6.39	0.0	0.3	27.69	30.250000
4	0.00	0.0	0.3	17.80	16.716667

4.2.4 Outliers

Call `df.info()` to inspect the columns and decide which ones to check for outliers.

```
[14]: ### YOUR CODE HERE ###
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            22699 non-null  int64
1   VendorID                              22699 non-null  int64
2   tpep_pickup_datetime                  22699 non-null  datetime64[ns]
3   tpep_dropoff_datetime                  22699 non-null  datetime64[ns]
4   passenger_count                       22699 non-null  int64
5   trip_distance                         22699 non-null  float64
6   RatecodeID                           22699 non-null  int64
7   store_and_fwd_flag                    22699 non-null  object
8   PULocationID                          22699 non-null  int64
9   DOLocationID                          22699 non-null  int64
10  payment_type                           22699 non-null  int64
11  fare_amount                           22699 non-null  float64
12  extra                                 22699 non-null  float64
13  mta_tax                               22699 non-null  float64
14  tip_amount                            22699 non-null  float64
15  tolls_amount                          22699 non-null  float64
16  improvement_surcharge                  22699 non-null  float64
17  total_amount                           22699 non-null  float64
18  duration                              22699 non-null  float64
dtypes: datetime64[ns](2), float64(9), int64(7), object(1)
memory usage: 3.3+ MB
```

Keeping in mind that many of the features will not be used to fit your model, the most important columns to check for outliers are likely to be: `* trip_distance * fare_amount * duration`

4.2.5 Task 2d. Box plots

Plot a box plot for each feature: `trip_distance`, `fare_amount`, `duration`.


```
[15]: ### YOUR CODE HERE ###

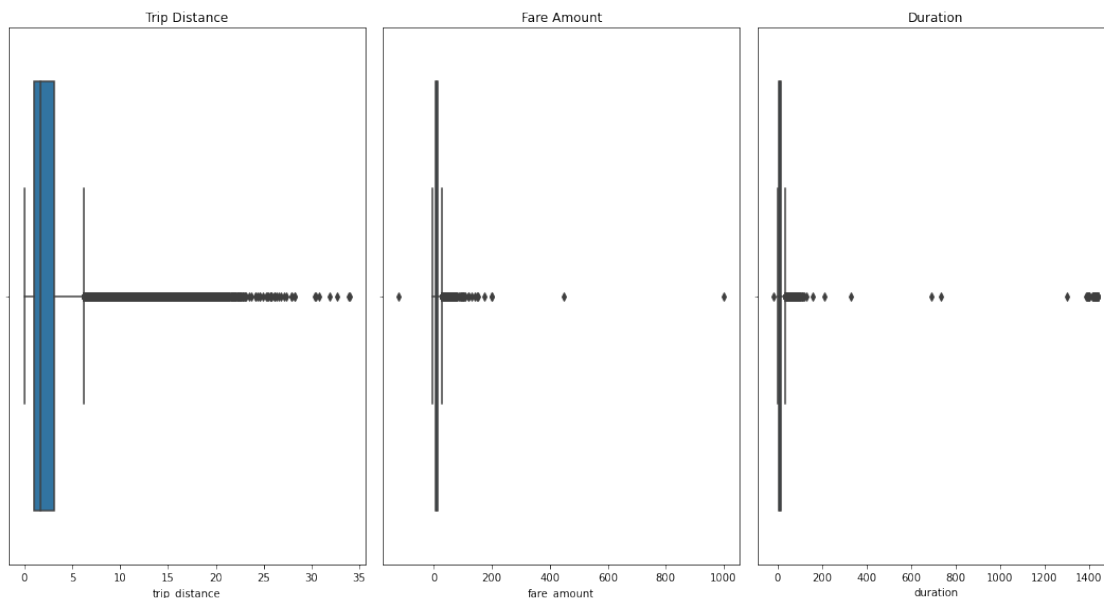
fig, axes = plt.subplots(1,3, figsize=(15,8))

axes[0].set_title("Trip Distance")
sns.boxplot(x=df['trip_distance'], ax=axes[0])

axes[1].set_title("Fare Amount")
sns.boxplot(x=df['fare_amount'], ax=axes[1])

axes[2].set_title("Duration")
sns.boxplot(x=df['duration'], ax=axes[2])

plt.tight_layout()
```



Questions: 1. Which variable(s) contains outliers?

2. Are the values in the `trip_distance` column unbelievable?

3. What about the lower end? Do distances, fares, and durations of 0 (or negative values) make sense?

1. All three variables contain outliers. Some are extreme, but others not so much.

2. It's 30 miles from the southern tip of Staten Island to the northern end of Manhattan and that's in a straight line. With this knowledge and the distribution of the values in this column, it's reasonable to leave these values alone and not alter them. However, the values for `fare_amount` and `duration` definitely seem to have problematic outliers on the higher end.

3. Probably not for the latter two, but for `trip_distance` it might be okay.

4.2.6 Task 2e. Imputations

trip_distance outliers You know from the summary statistics that there are trip distances of 0. Are these reflective of erroneous data, or are they very short trips that get rounded down?

To check, sort the column values, eliminate duplicates, and inspect the least 10 values. Are they rounded values or precise values?

```
[16]: # Are trip distances of 0 bad data or very short trips rounded down?
      ### YOUR CODE HERE ###
      df['trip_distance'].drop_duplicates().sort_values().head(10)
```

```
[16]: 128      0.00
      2985    0.01
      323    0.02
      3158   0.03
      1510   0.04
      10146  0.05
      4423   0.06
      922    0.07
      4623   0.08
      22035  0.09
      Name: trip_distance, dtype: float64
```

The distances are captured with a high degree of precision. However, it might be possible for trips to have distances of zero if a passenger summoned a taxi and then changed their mind. Besides, are there enough zero values in the data to pose a problem?

Calculate the count of rides where the `trip_distance` is zero.

```
[17]: ### YOUR CODE HERE ###
      (df[['trip_distance']] == 0).value_counts()
```

```
[17]: trip_distance
      False      22551
      True       148
      dtype: int64
```

fare_amount outliers

```
[18]: ### YOUR CODE HERE ###
      df['fare_amount'].describe()
```

```
[18]: count      22699.000000
      mean        13.026629
      std         13.243791
```

```

min          -120.000000
25%           6.500000
50%           9.500000
75%          14.500000
max          999.990000
Name: fare_amount, dtype: float64

```

Question: What do you notice about the values in the `fare_amount` column?

The range of values in the `fare_amount` column is large and the extremes don't make much sense.

- **Low values:** Negative values are problematic. Values of zero could be legitimate if the taxi logged a trip that was immediately canceled.
- **High values:** The maximum fare amount in this dataset is nearly \\$1,000, which seems very unlikely. High values for this feature can be capped based on intuition and statistics. The interquartile range (IQR) is \\$8. The standard formula of $Q3 + (1.5 * IQR)$ yields \$26.50. That doesn't seem appropriate for the maximum fare cap. In this case, we'll use a factor of 6, which results in a cap of \$62.50.

Impute values less than \$0 with 0.

```

[19]: # Impute values less than $0 with 0
      ### YOUR CODE HERE ###
      df.loc[df['fare_amount'] < 0, 'fare_amount'] = 0
      df.fare_amount.min()

```

[19]: 0.0

Now impute the maximum value as $Q3 + (6 * IQR)$.

```

[20]: ### YOUR CODE HERE ###
      def outlier_imputer(column_list, iqr_factor):

          """
          Impute upper-limit values in specified columns based on their interquartile_
          →range.

          Arguments:
              column_list: A list of columns to iterate over
              iqr_factor: A number representing x in the formula:
                           $Q3 + (x * IQR)$ . Used to determine maximum threshold,
                          beyond which a point is considered an outlier.

          The IQR is computed for each column in column_list and values exceeding
          the upper threshold for each column are imputed with the upper threshold_
          →value.
          """
      ### YOUR CODE HERE ###

```

```

for col in column_list:
    # Reassign minimum to zero
    ### YOUR CODE HERE ###

    df.loc[df[col] < 0, col] = 0

    # Calculate upper threshold
    ### YOUR CODE HERE ###

    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    upper_threshold = q3 + (iqr_factor * iqr)

    print("column:", col)
    print("iqr factor:", iqr_factor)

    print("q1:", q1)
    print("q3:", q3)
    print("iqr:", iqr)

    print("upper threshold : ", upper_threshold)
    # Reassign values > threshold to threshold
    ### YOUR CODE HERE ###

    df.loc[df[col] > upper_threshold, col] = upper_threshold
    print()
    print(df[col].describe())
    print()

outlier_imputer(['fare_amount'], 6)

```

```

column: fare_amount
iqr factor: 6
q1: 6.5
q3: 14.5
iqr: 8.0
upper threshold : 62.5

```

```

count    22699.000000
mean      12.897913
std       10.541137
min        0.000000
25%        6.500000
50%        9.500000
75%       14.500000

```

```
max          62.500000
Name: fare_amount, dtype: float64
```

duration outliers

```
[21]: # Call .describe() for duration outliers
      ### YOUR CODE HERE ###
      outlier_imputer(['duration'], 6)
```

```
column: duration
iqr factor: 6
q1: 6.65
q3: 18.383333333333333
iqr: 11.733333333333333
upper threshold : 88.78333333333333
```

```
count      22699.000000
mean        14.460555
std         11.947043
min          0.000000
25%          6.650000
50%         11.183333
75%         18.383333
max         88.783333
Name: duration, dtype: float64
```

The `duration` column has problematic values at both the lower and upper extremities.

- **Low values:** There should be no values that represent negative time. Impute all negative durations with 0.
- **High values:** Impute high values the same way you imputed the high-end outliers for fares: $Q3 + (6 * IQR)$.

```
[43]: # Impute a 0 for any negative values
      ### YOUR CODE HERE ###
      #already done
```

```
[44]: # Impute the high outliers
      ### YOUR CODE HERE ###
      #already done
```

4.2.7 Task 3a. Feature engineering

Create `mean_distance` column When deployed, the model will not know the duration of a trip until after the trip occurs, so you cannot train a model that uses this feature. However, you can use the statistics of trips you *do* know to generalize about ones you do not know.

In this step, create a column called `mean_distance` that captures the mean distance for each group of trips that share pickup and dropoff points.

For example, if your data were:

```
|Trip|Start|End|Distance| |-:|:-:|:-:| | 1 | A | B | 1 | | 2 | C | D | 2 | | 3 | A | B | 1.5 | | 4 | D | C | 3 |
```

The results should be:

A -> B: 1.25 miles

C -> D: 2 miles

D -> C: 3 miles

Notice that C -> D is not the same as D -> C. All trips that share a unique pair of start and end points get grouped and averaged.

Then, a new column `mean_distance` will be added where the value at each row is the average for all trips with those pickup and dropoff locations:

Trip	Start	End	Distance	mean_distance
1	A	B	1	1.25
2	C	D	2	2
3	A	B	1.5	1.25
4	D	C	3	3

Begin by creating a helper column called `pickup_dropoff`, which contains the unique combination of pickup and dropoff location IDs for each row.

One way to do this is to convert the pickup and dropoff location IDs to strings and join them, separated by a space. The space is to ensure that, for example, a trip with pickup/dropoff points of 12 & 151 gets encoded differently than a trip with points 121 & 51.

So, the new column would look like this:

Trip	Start	End	pickup_dropoff
1	A	B	'A B'
2	C	D	'C D'
3	A	B	'A B'
4	D	C	'D C'

```
[22]: # Create `pickup_dropoff` column
      ### YOUR CODE HERE ###
      df['pickup_dropoff'] = df['PULocationID'].astype(str) + ' ' +
      ↪df['DOLocationID'].astype(str)
      df['pickup_dropoff'].head()
```

```
[22]: 0    100 231
      1    186 43
      2    262 236
      3    188 97
      4     4 112
      Name: pickup_dropoff, dtype: object
```

Now, use a `groupby()` statement to group each row by the new `pickup_dropoff` column, compute the mean, and capture the values only in the `trip_distance` column. Assign the results to a variable named `grouped`.

```
[23]: ### YOUR CODE HERE ###
      grouped = df.groupby('pickup_dropoff')[['trip_distance']].
           ↳mean(numeric_only=True)
      grouped.head
```

```
[23]: <bound method NDFrame.head of                                trip_distance
      pickup_dropoff
      1 1                                2.433333
      10 148                             15.700000
      100 1                             16.890000
      100 100                             0.253333
      100 107                             1.180000
      ...
      97 65                                0.500000
      97 66                                1.400000
      97 80                                3.840000
      97 90                                4.420000
      97 97                                1.006667

      [4172 rows x 1 columns]>
```

`grouped` is an object of the `DataFrame` class.

1. Convert it to a dictionary using the `to_dict()` method. Assign the results to a variable called `grouped_dict`. This will result in a dictionary with a key of `trip_distance` whose values are another dictionary. The inner dictionary's keys are pickup/dropoff points and its values are mean distances. This is the information you want.

Example:

```
grouped_dict = {'trip_distance': {'A B': 1.25, 'C D': 2, 'D C': 3}}
```

2. Reassign the `grouped_dict` dictionary so it contains only the inner dictionary. In other words, get rid of `trip_distance` as a key, so:

Example:

```
grouped_dict = {'A B': 1.25, 'C D': 2, 'D C': 3}
```

```
[24]: # 1. Convert `grouped` to a dictionary
      ### YOUR CODE HERE ###
      grouped_dict = grouped.to_dict()

      # 2. Reassign to only contain the inner dictionary
      ### YOUR CODE HERE ###
      grouped_dict = grouped_dict['trip_distance']
```

1. Create a `mean_distance` column that is a copy of the `pickup_dropoff` helper column.
2. Use the `map()` method on the `mean_distance` series. Pass `grouped_dict` as its argument. Reassign the result back to the `mean_distance` series. When you pass a dictionary to the `Series.map()` method, it will replace the data in the series where that data matches the dictionary's keys. The values that get imputed are the values of the dictionary.

Example:

```
df['mean_distance']
```

mean_distance
'A B'
'C D'
'A B'
'D C'
'E F'

```
grouped_dict = {'A B': 1.25, 'C D': 2, 'D C': 3}
df['mean_distance'] = df['mean_distance'].map(grouped_dict)
df['mean_distance']
```

mean_distance
1.25
2
1.25
3
NaN

When used this way, the `map()` Series method is very similar to `replace()`, however, note that `map()` will impute `NaN` for any values in the series that do not have a corresponding key in the mapping dictionary, so be careful.

```
[25]: # 1. Create a mean_distance column that is a copy of the pickup_dropoff helper_
      ↪ column
      ### YOUR CODE HERE ###
      df['mean_distance'] = df['pickup_dropoff']

      # 2. Map `grouped_dict` to the `mean_distance` column
```



```

### YOUR CODE HERE ###

df['mean_distance'] = df['mean_distance'].map(grouped_dict)

# Confirm that it worked
### YOUR CODE HERE ###
df[(df['PULocationID']==100) & (df['DOLocationID']==231)][['mean_distance']]

```

```

[25]:      mean_distance
0          3.521667
4909       3.521667
16636      3.521667
18134      3.521667
19761      3.521667
20581      3.521667

```

Create mean_duration column Repeat the process used to create the mean_distance column to create a mean_duration column.

```

[26]: ### YOUR CODE HERE ###

grouped = df.groupby('pickup_dropoff').mean(numeric_only=True)[['duration']]
grouped

# Create a dictionary where keys are unique pickup_dropoffs and values are
# mean trip duration for all trips with those pickup_dropoff combos
### YOUR CODE HERE ###

grouped_dict = grouped.to_dict()
grouped_dict = grouped_dict['duration']

df['mean_duration'] = df['pickup_dropoff']
df['mean_duration'] = df['mean_duration'].map(grouped_dict)

# Confirm that it worked
### YOUR CODE HERE ###

df[(df['PULocationID']==100) & (df['DOLocationID']==231)][['mean_duration']]

```

```

[26]:      mean_duration
0          22.847222
4909       22.847222
16636      22.847222
18134      22.847222
19761      22.847222
20581      22.847222

```

Create day and month columns Create two new columns, `day` (name of day) and `month` (name of month) by extracting the relevant information from the `tpep_pickup_datetime` column.

```
[27]: # Create 'day' col
      ### YOUR CODE HERE ###

      df['day'] = df['tpep_pickup_datetime'].dt.day_name().str.lower()

      # Create 'month' col
      ### YOUR CODE HERE ###

      df['month'] = df['tpep_pickup_datetime'].dt.strftime('%b').str.lower()
```

Create rush_hour column Define rush hour as: * Any weekday (not Saturday or Sunday) AND * Either from 06:00–10:00 or from 16:00–20:00

Create a binary `rush_hour` column that contains a 1 if the ride was during rush hour and a 0 if it was not.

```
[28]: # Create 'rush_hour' col
      ### YOUR CODE HERE ###

      df['rush_hour'] = df['tpep_pickup_datetime'].dt.hour

      # If day is Saturday or Sunday, impute 0 in `rush_hour` column
      ### YOUR CODE HERE ###

      df.loc[df['day'].isin(['saturday', 'sunday']), 'rush_hour'] = 0
```

```
[29]: ### YOUR CODE HERE ###
def rush_hourizer(hour):
    if 6 <= hour['rush_hour'] < 10:
        val = 1
    elif 16 <= hour['rush_hour'] <= 20:
        val = 1
    else:
        val = 0
    return val
```

```
[30]: # Apply the `rush_hourizer()` function to the new column
      ### YOUR CODE HERE ###
      df.loc[(df['day'] != 'saturday') & (df['day'] != 'sunday'), 'rush_hour'] = df.
      ↪apply(rush_hourizer, axis=1)
      df.head()
```

```
[30]: Unnamed: 0  VendorID  tpep_pickup_datetime  tpep_dropoff_datetime  \
0      24870114         2  2017-03-25 08:55:43    2017-03-25 09:09:47
1      35634249         1  2017-04-11 14:53:28    2017-04-11 15:19:58
```

2	106203690	1	2017-12-15 07:26:56	2017-12-15 07:34:08
3	38942136	2	2017-05-07 13:17:59	2017-05-07 13:48:14
4	30841670	2	2017-04-15 23:32:20	2017-04-15 23:49:03

	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	\
0	6	3.34	1	N	
1	1	1.80	1	N	
2	1	1.00	1	N	
3	1	3.70	1	N	
4	1	4.37	1	N	

	PULocationID	DOLocationID	...	tolls_amount	improvement_surcharge	\
0	100	231	...	0.0	0.3	
1	186	43	...	0.0	0.3	
2	262	236	...	0.0	0.3	
3	188	97	...	0.0	0.3	
4	4	112	...	0.0	0.3	

	total_amount	duration	pickup_dropoff	mean_distance	mean_duration	\
0	16.56	14.066667	100 231	3.521667	22.847222	
1	20.80	26.500000	186 43	3.108889	24.470370	
2	8.75	7.200000	262 236	0.881429	7.250000	
3	27.69	30.250000	188 97	3.700000	30.250000	
4	17.80	16.716667	4 112	4.435000	14.616667	

	day	month	rush_hour
0	saturday	mar	0
1	tuesday	apr	0
2	friday	dec	1
3	sunday	may	0
4	saturday	apr	0

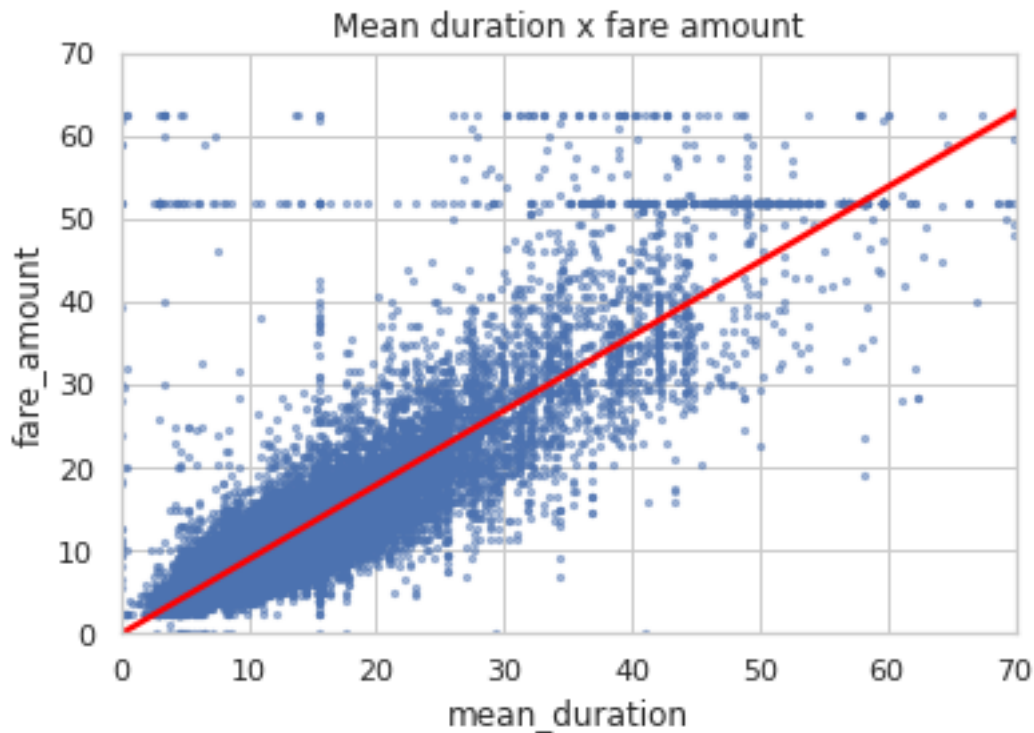
[5 rows x 25 columns]

4.2.8 Task 4. Scatter plot

Create a scatterplot to visualize the relationship between `mean_duration` and `fare_amount`.

```
[31]: # Create a scatterplot to visualize the relationship between variables of
      ↪ interest
      ### YOUR CODE HERE ###
      sns.set(style='whitegrid')
      sns.regplot(x=df['mean_duration'], y=df['fare_amount'],
                  scatter_kws={'alpha':0.5, 's':5},
                  line_kws={'color':'red'})
      plt.ylim(0, 70)
```

```
plt.xlim(0, 70)
plt.title('Mean duration x fare amount')
plt.show()
```



The `mean_duration` variable correlates with the target variable. But what are the horizontal lines around fare amounts of 52 dollars and 63 dollars? What are the values and how many are there?

You know what one of the lines represents. 62 dollars and 50 cents is the maximum that was imputed for outliers, so all former outliers will now have fare amounts of \$62.50. What is the other line?

Check the value of the rides in the second horizontal line in the scatter plot.

```
[32]: ### YOUR CODE HERE ###
(df[['fare_amount']] > 50).value_counts().head()
```

```
[32]: fare_amount
False      22014
True         685
dtype: int64
```

Examine the first 30 of these trips.

```
[33]: # Set pandas to display all columns
      ### YOUR CODE HERE ###
      pd.set_option('display.max.columns', None)
      df[df['fare_amount'] == 52].head(20)
```

```
[33]: Unnamed: 0  VendorID  tpep_pickup_datetime  tpep_dropoff_datetime  \
11      18600059          2  2017-03-05 19:15:30  2017-03-05 19:52:18
110     47959795          1  2017-06-03 14:24:57  2017-06-03 15:31:48
161     95729204          2  2017-11-11 20:16:16  2017-11-11 20:17:14
247    103404868          2  2017-12-06 23:37:08  2017-12-07 00:06:19
379     80479432          2  2017-09-24 23:45:45  2017-09-25 00:15:14
388     16226157          1  2017-02-28 18:30:05  2017-02-28 19:09:55
406     55253442          2  2017-06-05 12:51:58  2017-06-05 13:07:35
449     65900029          2  2017-08-03 22:47:14  2017-08-03 23:32:41
468     80904240          2  2017-09-26 13:48:26  2017-09-26 14:31:17
520     33706214          2  2017-04-23 21:34:48  2017-04-23 22:46:23
569     99259872          2  2017-11-22 21:31:32  2017-11-22 22:00:25
572     61050418          2  2017-07-18 13:29:06  2017-07-18 13:29:19
586     54444647          2  2017-06-26 13:39:12  2017-06-26 14:34:54
692     94424289          2  2017-11-07 22:15:00  2017-11-07 22:45:32
717    103094220          1  2017-12-06 05:19:50  2017-12-06 05:53:52
719     66115834          1  2017-08-04 17:53:34  2017-08-04 18:50:56
782     55934137          2  2017-06-09 09:31:25  2017-06-09 10:24:10
816     13731926          2  2017-02-21 06:11:03  2017-02-21 06:59:39
818     52277743          2  2017-06-20 08:15:18  2017-06-20 10:24:37
835      2684305          2  2017-01-10 22:29:47  2017-01-10 23:06:46

      passenger_count  trip_distance  RatecodeID  store_and_fwd_flag  \
11                   2          18.90          2                   N
110                  1          18.00          2                   N
161                  1           0.23          2                   N
247                  1          18.93          2                   N
379                  1          17.99          2                   N
388                  1          18.40          2                   N
406                  1           4.73          2                   N
449                  2          18.21          2                   N
468                  1          17.27          2                   N
520                  6          18.34          2                   N
569                  1          18.65          2                   N
572                  1           0.00          2                   N
586                  1          17.76          2                   N
692                  2          16.97          2                   N
717                  1          20.80          2                   N
719                  1          21.60          2                   N
782                  2          18.81          2                   N
816                  5          16.94          2                   N
818                  1          17.77          2                   N
```

835		1	18.57	2		N
-----	--	---	-------	---	--	---

	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	\
11	236	132	1	52.0	0.0	0.5	
110	132	163	1	52.0	0.0	0.5	
161	132	132	2	52.0	0.0	0.5	
247	132	79	2	52.0	0.0	0.5	
379	132	234	1	52.0	0.0	0.5	
388	132	48	2	52.0	4.5	0.5	
406	228	88	2	52.0	0.0	0.5	
449	132	48	2	52.0	0.0	0.5	
468	186	132	2	52.0	0.0	0.5	
520	132	148	1	52.0	0.0	0.5	
569	132	144	1	52.0	0.0	0.5	
572	230	161	1	52.0	0.0	0.5	
586	211	132	1	52.0	0.0	0.5	
692	132	170	1	52.0	0.0	0.5	
717	132	239	1	52.0	0.0	0.5	
719	264	264	1	52.0	4.5	0.5	
782	163	132	1	52.0	0.0	0.5	
816	132	170	1	52.0	0.0	0.5	
818	132	246	1	52.0	0.0	0.5	
835	132	48	1	52.0	0.0	0.5	

	tip_amount	tolls_amount	improvement_surcharge	total_amount	duration	\
11	14.58	5.54	0.3	72.92	36.800000	
110	0.00	0.00	0.3	52.80	66.850000	
161	0.00	0.00	0.3	52.80	0.966667	
247	0.00	0.00	0.3	52.80	29.183333	
379	14.64	5.76	0.3	73.20	29.483333	
388	0.00	5.54	0.3	62.84	39.833333	
406	0.00	5.76	0.3	58.56	15.616667	
449	0.00	5.76	0.3	58.56	45.450000	
468	0.00	5.76	0.3	58.56	42.850000	
520	5.00	0.00	0.3	57.80	71.583333	
569	10.56	0.00	0.3	63.36	28.883333	
572	11.71	5.76	0.3	70.27	0.216667	
586	11.71	5.76	0.3	70.27	55.700000	
692	11.71	5.76	0.3	70.27	30.533333	
717	5.85	5.76	0.3	64.41	34.033333	
719	12.60	5.76	0.3	75.66	57.366667	
782	13.20	0.00	0.3	66.00	52.750000	
816	2.00	5.54	0.3	60.34	48.600000	
818	11.71	5.76	0.3	70.27	88.783333	
835	13.20	0.00	0.3	66.00	36.983333	

pickup_dropoff	mean_distance	mean_duration	day	month	rush_hour
----------------	---------------	---------------	-----	-------	-----------

11	236 132	19.211667	40.500000	sunday	mar	0
110	132 163	19.229000	52.941667	saturday	jun	0
161	132 132	2.255862	3.021839	saturday	nov	0
247	132 79	19.431667	47.275000	wednesday	dec	0
379	132 234	17.654000	49.833333	sunday	sep	0
388	132 48	18.761905	58.246032	tuesday	feb	1
406	228 88	4.730000	15.616667	monday	jun	0
449	132 48	18.761905	58.246032	thursday	aug	0
468	186 132	17.096000	42.920000	tuesday	sep	0
520	132 148	17.994286	46.340476	sunday	apr	0
569	132 144	18.537500	37.000000	wednesday	nov	0
572	230 161	0.685484	7.965591	tuesday	jul	0
586	211 132	16.580000	61.691667	monday	jun	0
692	132 170	17.203000	37.113333	tuesday	nov	0
717	132 239	20.901250	44.862500	wednesday	dec	0
719	264 264	3.191516	15.618773	friday	aug	1
782	163 132	17.275833	52.338889	friday	jun	1
816	132 170	17.203000	37.113333	tuesday	feb	1
818	132 246	18.515000	66.316667	tuesday	jun	1
835	132 48	18.761905	58.246032	tuesday	jan	0

Question: What do you notice about the first 30 trips?

It seems that almost all of the trips in the first 30 rows where the fare amount was \$52 either begin or end at location 132, and all of them have a `RatecodeID` of 2.

There is no readily apparent reason why `PULocation` 132 should have so many fares of 52 dollars. They seem to occur on all different days, at different times, with both vendors, in all months. However, there are many toll amounts of \$5.76 and \$5.54. This would seem to indicate that location 132 is in an area that frequently requires tolls to get to and from. It's likely this is an airport.

The data dictionary says that `RatecodeID` of 2 indicates trips for JFK, which is John F. Kennedy International Airport. A quick Google search for “new york city taxi flat rate \$52” indicates that in 2017 (the year that this data was collected) there was indeed a flat fare for taxi trips between JFK airport (in Queens) and Manhattan.

Because `RatecodeID` is known from the data dictionary, the values for this rate code can be imputed back into the data after the model makes its predictions. This way you know that those data points will always be correct.

4.2.9 Task 5. Isolate modeling variables

Drop features that are redundant, irrelevant, or that will not be available in a deployed environment.

```
[45]: ### YOUR CODE HERE ###
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 22699 entries, 0 to 22698
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Unnamed: 0                            22699 non-null  int64
1   VendorID                              22699 non-null  int64
2   tpep_pickup_datetime                  22699 non-null  datetime64[ns]
3   tpep_dropoff_datetime                 22699 non-null  datetime64[ns]
4   passenger_count                       22699 non-null  int64
5   trip_distance                         22699 non-null  float64
6   RatecodeID                           22699 non-null  int64
7   store_and_fwd_flag                   22699 non-null  object
8   PULocationID                         22699 non-null  int64
9   DOLocationID                         22699 non-null  int64
10  payment_type                          22699 non-null  int64
11  fare_amount                           22699 non-null  float64
12  extra                                 22699 non-null  float64
13  mta_tax                               22699 non-null  float64
14  tip_amount                            22699 non-null  float64
15  tolls_amount                          22699 non-null  float64
16  improvement_surcharge                 22699 non-null  float64
17  total_amount                          22699 non-null  float64
18  duration                              22699 non-null  float64
19  pickup_dropoff                       22699 non-null  object
20  mean_distance                         22699 non-null  float64
21  mean_duration                         22699 non-null  float64
22  day                                   22699 non-null  object
23  month                                 22699 non-null  object
24  rush_hour                             22699 non-null  int64
dtypes: datetime64[ns](2), float64(11), int64(8), object(4)
memory usage: 4.3+ MB

```

```

[34]: ### YOUR CODE HERE ###
df2 = df.copy()
df2 = df2.drop(['Unnamed: 0', 'tpep_dropoff_datetime', 'tpep_pickup_datetime',
               'trip_distance', 'RatecodeID', 'store_and_fwd_flag',
               ↪ 'PULocationID', 'DOLocationID',
               'payment_type', 'extra', 'mta_tax', 'tip_amount',
               ↪ 'tolls_amount', 'improvement_surcharge',
               'total_amount', 'tpep_dropoff_datetime', 'tpep_pickup_datetime',
               ↪ 'duration',
               'pickup_dropoff', 'day', 'month'
               ], axis=1)
df2.info()

```

```

<class 'pandas.core.frame.DataFrame'>

```



```

RangeIndex: 22699 entries, 0 to 22698
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   VendorID              22699 non-null  int64
 1   passenger_count       22699 non-null  int64
 2   fare_amount           22699 non-null  float64
 3   mean_distance         22699 non-null  float64
 4   mean_duration         22699 non-null  float64
 5   rush_hour             22699 non-null  int64
dtypes: float64(3), int64(3)
memory usage: 1.0 MB

```

4.2.10 Task 6. Pair plot

Create a pairplot to visualize pairwise relationships between `fare_amount`, `mean_duration`, and `mean_distance`.

```

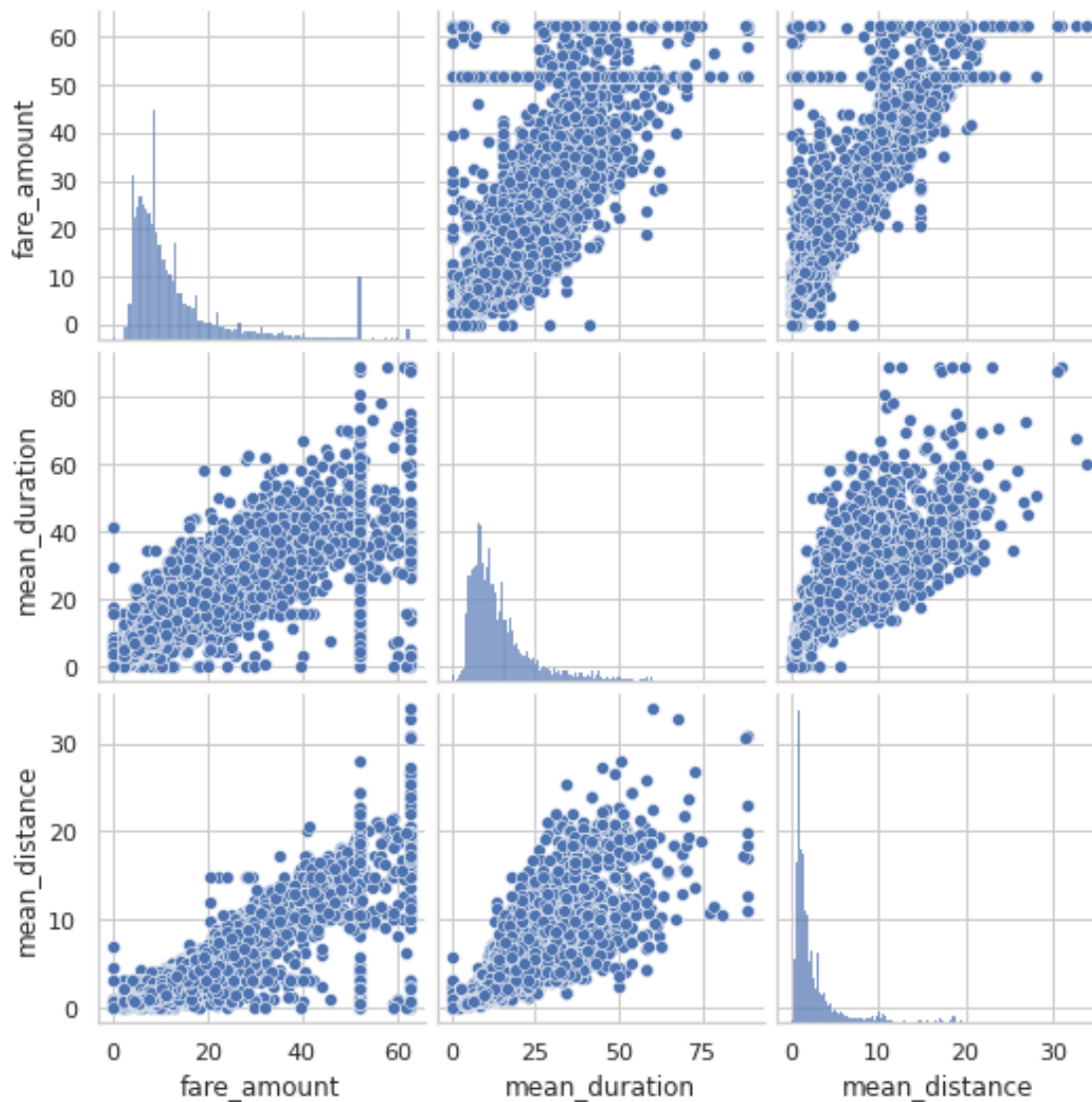
[35]: # Create a pairplot to visualize pairwise relationships between variables in
      → the data
      ### YOUR CODE HERE ###
      sns.pairplot(df[['fare_amount', 'mean_duration', 'mean_distance']])

```

```

[35]: <seaborn.axisgrid.PairGrid at 0x7aca442a8ad0>

```



These variables all show linear correlation with each other. Investigate this further.

4.2.11 Task 7. Identify correlations

Next, code a correlation matrix to help determine most correlated variables.

```
[36]: # Correlation matrix to help determine most correlated variables
      ### YOUR CODE HERE ###
      df2.corr(method='pearson')
```

```
[36]:
```

	VendorID	passenger_count	fare_amount	mean_distance	\
VendorID	1.000000	0.266463	0.001045	0.004741	
passenger_count	0.266463	1.000000	0.014942	0.013428	

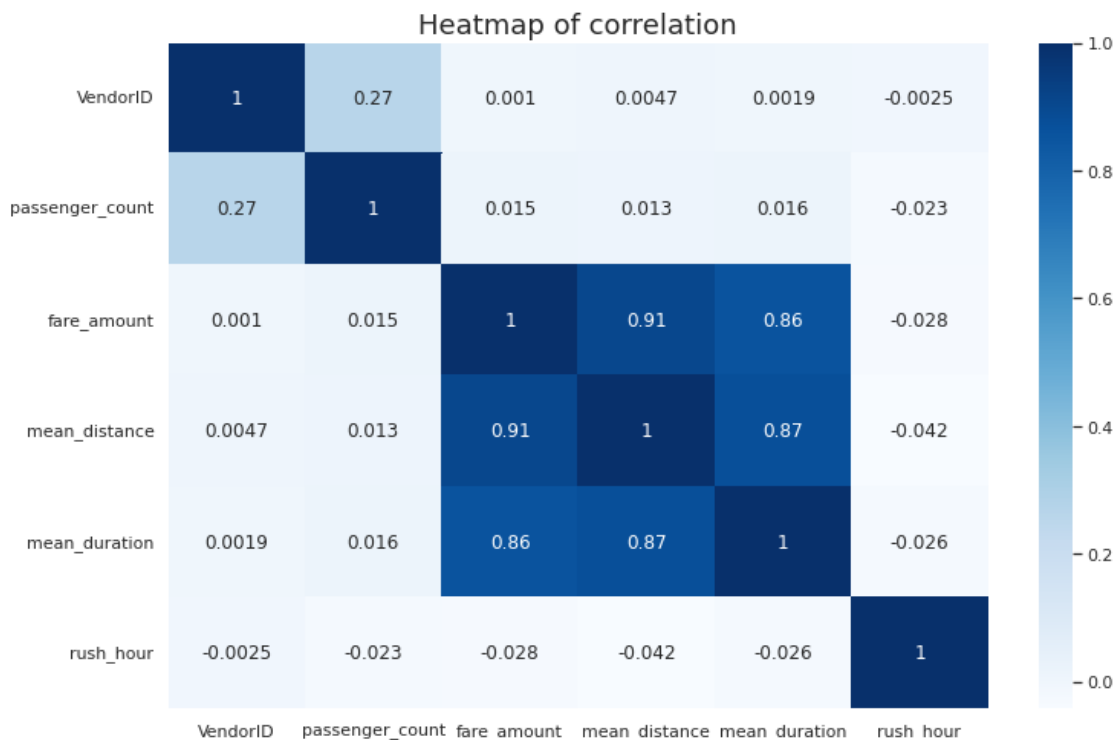
fare_amount	0.001045	0.014942	1.000000	0.910185
mean_distance	0.004741	0.013428	0.910185	1.000000
mean_duration	0.001876	0.015852	0.859105	0.874864
rush_hour	-0.002522	-0.023380	-0.027583	-0.042347

	mean_duration	rush_hour
VendorID	0.001876	-0.002522
passenger_count	0.015852	-0.023380
fare_amount	0.859105	-0.027583
mean_distance	0.874864	-0.042347
mean_duration	1.000000	-0.025724
rush_hour	-0.025724	1.000000

Visualize a correlation heatmap of the data.

```
[37]: # Create correlation heatmap
      ### YOUR CODE HERE ###
      plt.figure(figsize=(12,8))
      sns.heatmap(df2.corr(method='pearson'), annot=True, cmap='Blues')
      plt.title("Heatmap of correlation", fontsize = 18)
```

```
[37]: Text(0.5, 1.0, 'Heatmap of correlation')
```



Question: Which variable(s) are correlated with the target variable of `fare_amount`?

mean_duration and mean_distance are both highly correlated with the target variable fare_amount. They're also both correlated with each other, with a Pearson correlation of 0.87.

Recall that highly correlated predictor variables can be bad for linear regression models when you want to be able to draw statistical inferences about the data from the model. However, correlated predictor variables can still be used to create an accurate predictor if the prediction itself is more important than using the model as a tool to learn about your data.

This model will predict fare_amount, which will be used as a predictor variable in machine learning models. Therefore, try modeling with both variables even though they are correlated.

4.3 PACE: Construct

After analysis and deriving variables with close relationships, it is time to begin constructing the model. Consider the questions in your PACE Strategy Document to reflect on the Construct stage.

4.3.1 Task 8a. Split data into outcome variable and features

```
[56]: ### YOUR CODE HERE ###  
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 22699 entries, 0 to 22698  
Data columns (total 6 columns):  
#   Column                Non-Null Count  Dtype  
---  ---  
0   VendorID              22699 non-null  int64  
1   passenger_count       22699 non-null  int64  
2   fare_amount           22699 non-null  float64  
3   mean_distance         22699 non-null  float64  
4   mean_duration         22699 non-null  float64  
5   rush_hour             22699 non-null  int64  
dtypes: float64(3), int64(3)  
memory usage: 1.0 MB
```

Set your X and y variables. X represents the features and y represents the outcome (target) variable.

```
[38]: # Remove the target column from the features  
# X = df2.drop(columns='fare_amount')  
### YOUR CODE HERE ###  
  
X = df2.drop(columns='fare_amount')  
  
# Set y variable  
### YOUR CODE HERE ###  
  
y = df2[['fare_amount']]
```

```
# Display first few rows
### YOUR CODE HERE ###
display(X.head())
display(y.head())
```

	VendorID	passenger_count	mean_distance	mean_duration	rush_hour
0	2	6	3.521667	22.847222	0
1	1	1	3.108889	24.470370	0
2	1	1	0.881429	7.250000	1
3	2	1	3.700000	30.250000	0
4	2	1	4.435000	14.616667	0

	fare_amount
0	13.0
1	16.0
2	6.5
3	20.5
4	16.5

4.3.2 Task 8b. Pre-process data

Dummy encode categorical variables

```
[39]: # Convert VendorID to string
### YOUR CODE HERE ###

X['VendorID'] = X['VendorID'].astype(str)

# Get dummies
### YOUR CODE HERE ###

X = pd.get_dummies(X, drop_first=True)
X.head()
```

```
[39]:
```

	passenger_count	mean_distance	mean_duration	rush_hour	VendorID_2
0	6	3.521667	22.847222	0	1
1	1	3.108889	24.470370	0	0
2	1	0.881429	7.250000	1	0
3	1	3.700000	30.250000	0	1
4	1	4.435000	14.616667	0	1

4.3.3 Split data into training and test sets

Create training and testing sets. The test set should contain 20% of the total samples. Set `random_state=0`.

```
[40]: # Create training and testing sets
      ##### YOUR CODE HERE #####
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size= 0.20,
      ↪random_state = 0)
```

4.3.4 Standardize the data

Use `StandardScaler()`, `fit()`, and `transform()` to standardize the `X_train` variables. Assign the results to a variable called `X_train_scaled`.

```
[41]: # Standardize the X variables
      ### YOUR CODE HERE ###
      scaler = StandardScaler().fit(X_train)
      X_train_scaled = scaler.transform(X_train)

      print('X_train scaled:', X_train_scaled)
```

```
X_train scaled: [[-0.50301524  0.8694684   0.17616665 -0.71641468  0.89286563]
 [-0.50301524 -0.60011281 -0.69829589  1.39583962  0.89286563]
 [ 0.27331093 -0.47829156 -0.57301906 -0.71641468 -1.11998936]
 ...
 [-0.50301524 -0.45121122 -0.6788917   -0.71641468 -1.11998936]
 [-0.50301524 -0.58944763 -0.85743597  1.39583962 -1.11998936]
 [ 1.82596329  0.83673851  1.13212101 -0.71641468  0.89286563]]
```

4.3.5 Fit the model

Instantiate your model and fit it to the training data.

```
[42]: # Fit your model to the training data
      ### YOUR CODE HERE ###
      lr = LinearRegression()
      lr.fit(X_train_scaled, y_train)
```

```
[42]: LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

4.3.6 Task 8c. Evaluate model

4.3.7 Train data

Evaluate your model performance by calculating the residual sum of squares and the explained variance score (R^2). Calculate the Mean Absolute Error, Mean Squared Error, and the Root Mean Squared Error.

```
[43]: # Evaluate the model performance on the training data
      ### YOUR CODE HERE ###

      r_sq = lr.score(X_train_scaled, y_train)
      print('Coefficient of determination:', r_sq)

      y_pred_train = lr.predict(X_train_scaled)
      print('R^2:', r2_score(y_train, y_pred_train))
      print('MAE:', mean_absolute_error(y_train, y_pred_train))
      print('MSE:', mean_squared_error(y_train, y_pred_train))
      print('RMSE:', np.sqrt(mean_squared_error(y_train, y_pred_train)))
```

```
Coefficient of determination: 0.8397674882008613
R^2: 0.8397674882008613
MAE: 2.1887939275819153
MSE: 17.898218963703183
RMSE: 4.230628672396477
```

4.3.8 Test data

Calculate the same metrics on the test data. Remember to scale the `X_test` data using the scaler that was fit to the training data. Do not refit the scaler to the testing data, just transform it. Call the results `X_test_scaled`.

```
[44]: # Scale the X_test data
      ### YOUR CODE HERE ###

      X_test_scaled = scaler.transform(X_test)

[45]: # Evaluate the model performance on the testing data
      ### YOUR CODE HERE ###

      r_sq_test = lr.score(X_test_scaled, y_test)
      print('Coefficient of determination:', r_sq_test)
      y_pred_test = lr.predict(X_test_scaled)
      print('R^2:', r2_score(y_test, y_pred_test))
      print('MAE:', mean_absolute_error(y_test, y_pred_test))
      print('MSE:', mean_squared_error(y_test, y_pred_test))
      print('RMSE:', np.sqrt(mean_squared_error(y_test, y_pred_test)))
```

Coefficient of determination: 0.8680805325784082
R²: 0.8680805325784082
MAE: 2.1355012690483255
MSE: 14.345792737892454
RMSE: 3.78758402387227

4.4 PACE: Execute

Consider the questions in your PACE Strategy Document to reflect on the Execute stage.

4.4.1 Task 9a. Results

Use the code cell below to get `actual`, `predicted`, and `residual` for the testing set, and store them as columns in a `results` dataframe.

```
[47]: # Create a `results` dataframe
      ### YOUR CODE HERE ###

      result = pd.DataFrame(data={
          'actual': y_test['fare_amount'],
          'predicted': y_pred_test.ravel()
      })
      result['residual'] = result['actual'] - result['predicted']
      result.head()
```

```
[47]:      actual  predicted  residual
      5818    14.0   12.384825    1.615175
      18134   28.0   16.345358   11.654642
      4655     5.5    6.754965   -1.254965
      7378    15.5   16.253895   -0.753895
      13914     9.5   10.565314   -1.065314
```

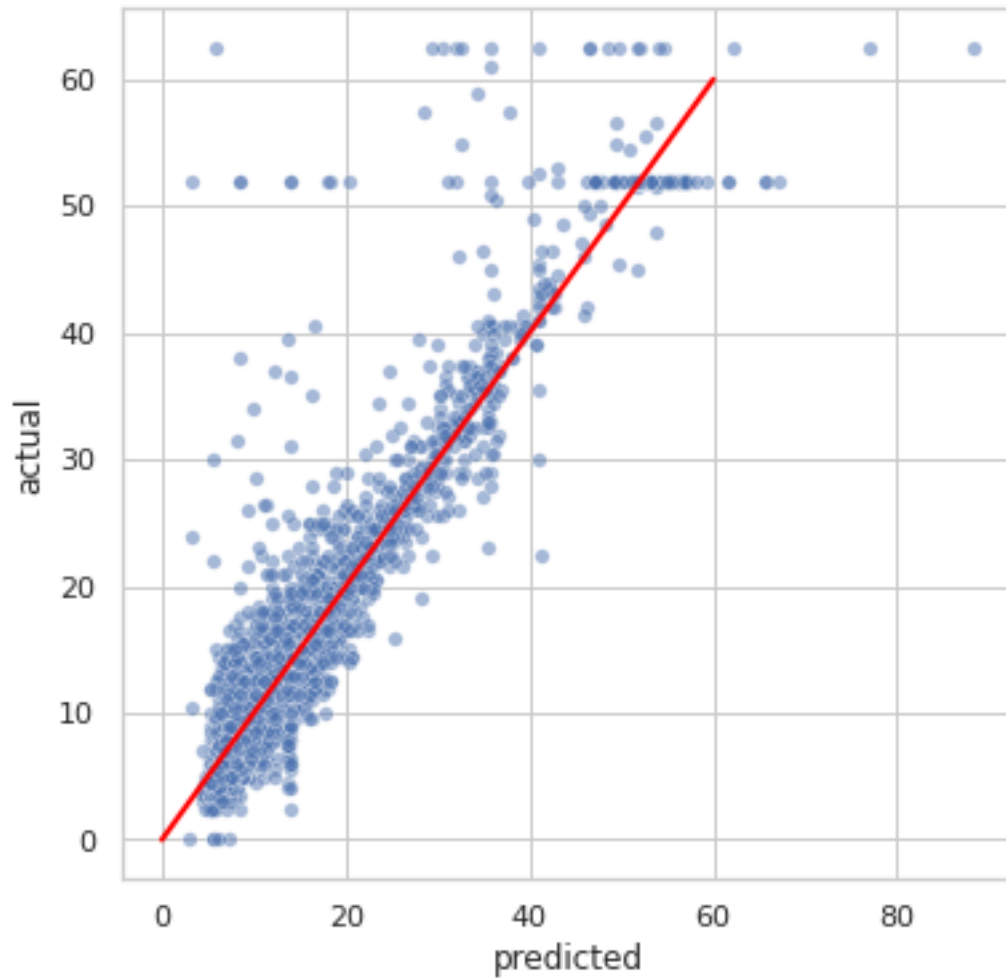
4.4.2 Task 9b. Visualize model results

Create a scatterplot to visualize actual vs. predicted.

```
[54]: # Create a scatterplot to visualize `predicted` over `actual`
      ### YOUR CODE HERE ###

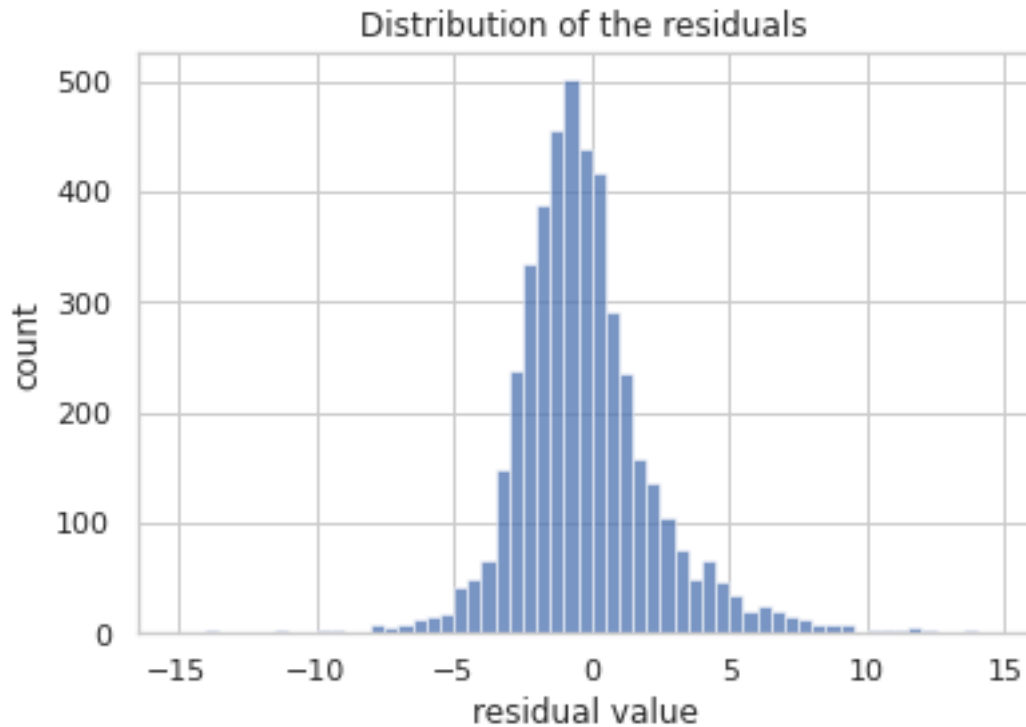
      fig, ax = plt.subplots(figsize=(6, 6))
      sns.set(style='whitegrid')
      sns.scatterplot(x='predicted', y='actual', data=result, alpha=0.5, s=30)
      plt.plot([0,60], [0,60], c='red', linewidth=2)
```

```
[54]: [ <matplotlib.lines.Line2D at 0x7aca3bc52a50> ]
```

Visualize the distribution of the residuals using a histogram.

```
[61]: # Visualize the distribution of the `residuals`  
      ### YOUR CODE HERE ###  
  
      sns.histplot(result['residual'], bins=np.arange(-15,15,.5))  
      plt.title('Distribution of the residuals')  
      plt.xlabel('residual value')  
      plt.ylabel('count');
```



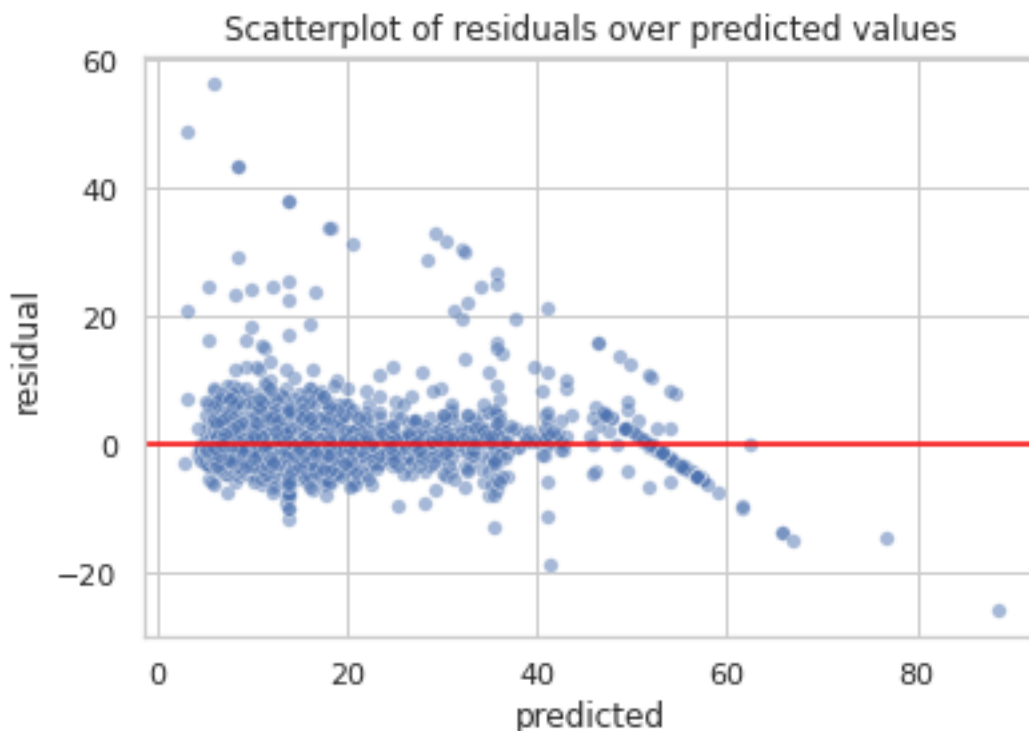
```
[59]: # Calculate residual mean
      ### YOUR CODE HERE ###
      result['residual'].mean()
```

```
[59]: -0.015237597152797327
```

Create a scatterplot of residuals over predicted.

```
[67]: # Create a scatterplot of `residuals` over `predicted`
      ### YOUR CODE HERE ###

      sns.set(style='whitegrid')
      sns.scatterplot(x='predicted', y='residual', data=result, alpha=0.5, s=30)
      plt.title('Scatterplot of residuals over predicted values')
      plt.axhline(0, c='red')
      plt.show()
```



4.4.3 Task 9c. Coefficients

Use the `coef_` attribute to get the model's coefficients. The coefficients are output in the order of the features that were used to train the model. Which feature had the greatest effect on trip fare?

```
[69]: # Output the model's coefficients
coefficients = pd.DataFrame(lr.coef_, columns=X.columns)
coefficients
```

```
[69]:    passenger_count  mean_distance  mean_duration  rush_hour  VendorID_2
0          0.029859         7.128958         2.815652    0.060334    -0.054531
```

What do these coefficients mean? How should they be interpreted?

The coefficients reveal that **mean_distance** was the feature with the greatest weight in the model's final prediction. Be careful here! A common misinterpretation is that for every mile traveled, the fare amount increases by a mean of $\$7.13$. This is incorrect. Remember, the data used to train the model was standardized with `StandardScaler()`. As such, the units are no longer miles. In other words, you cannot say "for every mile traveled...", as stated above. The correct interpretation of this coefficient is: controlling for other variables, *for every +1 change in standard deviation*, the fare amount increases by a mean of $\$7.13$.

Note also that because some highly correlated features were not removed, the confidence interval of this assessment is wider.

So, translate this back to miles instead of standard deviation (i.e., unscale the data).

1. Calculate the standard deviation of `mean_distance` in the `X_train` data.
2. Divide the coefficient (7.133867) by the result to yield a more intuitive interpretation.

4.4.4 Task 9d. Conclusion

1. What are the key takeaways from this notebook?
2. What results can be presented from this notebook?
 - Multiple linear regression is a powerful tool to estimate a dependent continuous variable from several independent variables.
 - Exploratory data analysis is useful for selecting both numeric and categorical features for multiple linear regression.
 - Fitting multiple linear regression models may require trial and error to select variables that fit an accurate model while maintaining model assumptions (or not, depending on your use case).

What results can be presented from this notebook?

- You can discuss meeting linear regression assumptions, and you can present the MAE and RMSE scores obtained from the model.

Congratulations! You’ve completed this lab. However, you may not notice a green check mark next to this item on Coursera’s platform. Please continue your progress regardless of the check mark. Just click on the “save” icon at the top of this notebook to ensure your work has been logged.